

- Simule todos os trechos de códigos apresentados. Modifique os valores das localizações de memória afetadas para verificar se o funcionamento está coerente.
- Faça um programa que some duas variáveis de 8 bits:

$$C = A + B$$

COM 8 BITS

```
.DSEG  
.ORG SRAM_START
```

```
A: .BYTE 1;  
B: .BYTE 1;  
C: .BYTE 1;
```

```
.CSEG  
start:
```

```
LDS R16, A  
LDS R17, B
```

```
ADD R16, R17 ; resultado foi guardado no r16
```

```
STS C, R16 ; salvamos no C
```

- Refaça o exercício anterior, mas considerando variáveis de 16 bits.
- Observações:
 - Considere as variáveis A, B e C armazenadas sequencialmente a partir do primeiro endereço da memória SRAM.
 - Para o exercício com variáveis de 16 bits, considere a ordenação dos bytes como *little-endian*¹, ou seja, o byte menos significativo é armazenado no menor endereço.

COM 16

```
.DSEG  
.ORG SRAM_START
```

```
A: .BYTE 2  
B: .BYTE 2  
C: .BYTE 2
```

```
.CSEG
```

start:

LDS R16, A ; AL

LDS R17, B ; BL

ADD R16, R17 ;AL + BL

STS C, R16; CL = AL+ BL

LDS R16, A+1 ; AH

LDS R17, B+1 ; BH

ADC R16, R17 ; AH + BH + CH

STS C+1, R16 ; CH = AH + BH + C

COM 16 USANDO REGISTRADORES DIFERENTES:

.DSEG

.ORG SRAM_START

A: .BYTE 2

B: .BYTE 2

C: .BYTE 2

.CSEG

start:

LDS R16, A ; AL

LDS R17, B ; BL

LDS R18, A+1; AH

LDS R19, B+1 ; BH

ADD R16, R17 ;AL + BL

ASC R18, R19

STS C, R16; CL = AL+ BL

STS C+1 , R18

rjmp start

- Faça um programa que some duas variáveis (A e B) e armazena o resultado em uma terceira (C):

C = A + B

- Observações:
 - Considere todas as variáveis de 32 bits.
 - Considere as variáveis A, B e C armazenadas sequencialmente a partir do primeiro endereço da memória SRAM.
 - Faça 4 implementações utilizando os seguintes modos de endereçamento: Direto, Indireto (sem incrementar), Indireto com Pós-incremento e Indireto com Deslocamento.

Entrega

- Arquivo texto (*.asm) com todos os 4 códigos.